

KnowAct: Evaluating Functional Theory of Mind through Active Knowledge-State Diagnosis

Anonymous authors
Paper under double-blind review

Abstract

An agent cannot adapt well to a user merely by answering the current request; it must also decide what it still needs to learn about that user. This creates a measurement problem. Static Theory-of-Mind benchmarks test inference from supplied evidence, whereas interactive benchmarks typically score downstream task or social outcomes. The former removes evidence acquisition from the tested policy, and the latter can reward success without an accurate user model.

We argue that functional user modeling should instead be evaluated as active latent-state identification: the target state must be explicit, acquiring evidence must consume a limited budget, and success must be attributable to visible interaction rather than hidden-state leakage or a favorable prior. KnowAct instantiates these requirements for knowledge-grounded interaction. A public domain graph defines a shared hypothesis space; a hidden user knowledge map defines the reconstruction target; and a tested agent chooses diagnostic questions before submitting a full-map estimate.

The benchmark separates simulator, agent, and evaluation contexts to address the tension between natural interaction and experimental control. A de-identified answer blueprint constrains what the simulator may express, while deterministic node-level scoring avoids an evaluator-model confound. The initial domain is statistical learning, grounded in an authoritative textbook. This paper develops the benchmark’s measurement argument and validation protocol; empirical validation remains future work.

1 Introduction

In collaborative interaction, the most informative action is often not an answer but a question. A tutor must locate the boundary of a learner’s understanding; a research assistant must discover which assumptions its collaborator already accepts; a decision-support agent must identify which uncertainty is blocking progress. In each case, assistance depends on an internal estimate of the user and on choosing an action that improves that estimate.

This is more demanding than describing a mental state after the relevant evidence has been supplied. The agent must decide which uncertainty matters, acquire evidence under a budget, revise its estimate, and let that estimate change what it does next. User modeling is therefore part of the control loop, not merely an explanatory annotation attached to a completed dialogue.

Existing evaluations illuminate different parts of this problem but do not identify the full loop. Machine Theory-of-Mind (ToM) benchmarks ask models to infer beliefs, intentions, or emotions from fixed narratives and conversations (Le et al., 2019; Gandhi et al., 2023; Kim et al., 2023; Xu et al., 2024; Chen et al., 2024b). Their controlled questions make inference measurable, yet the benchmark—not the tested model—chooses the evidence. The model is never accountable for asking an uninformative question or for failing to resolve an uncertainty.

Interactive social-agent environments restore action, but usually score a holistic outcome such as goal completion, persuasion, or task success (Zhou et al., 2024; Mou et al., 2025;

Hwang et al., 2026; Wang et al., 2026; Zhou et al., 2026). Such outcomes intentionally combine user modeling with planning, domain competence, language generation, and strategy. A successful trajectory may therefore reflect an accurate user model, a strong generic policy, environmental shortcuts, or some combination of them. The inference–application gap observed in explicit ToM evaluation makes this ambiguity consequential (Gu et al., 2024).

The resulting gap is one of measurement, not simply coverage. Static tasks isolate inference by removing evidence acquisition; interactive tasks restore action but often make the latent user state unobservable to the evaluator. Neither setting directly tests whether an agent can improve an explicit user model by choosing what evidence to request.

We study the following question:

Can an agent actively infer a user’s hidden knowledge state through limited natural-language interaction, and use that evolving estimate to choose subsequent diagnostic actions?

We call the target behavior functional ToM for knowledge-grounded interaction. Functional means that a user model earns credit through an infer–update–act loop, not through a verbal claim that the model understands the user. Knowledge-grounded narrows the latent state to reviewed, diagnosable concepts. This operational definition does not assert a human-like cognitive mechanism; it identifies behavior that a useful user model should support.

A valid test of this behavior requires three commitments. First, the latent state must be explicit enough to score directly; otherwise task success cannot establish that the user was modeled accurately. Second, evidence must be selected under a finite budget; otherwise an agent need not prioritize uncertainty. Third, the interaction must be auditable; otherwise leakage, unsupported inference, or simulator artifacts can masquerade as diagnosis.

KnowAct turns these commitments into a controlled benchmark. A public knowledge graph defines the shared hypothesis space, while a hidden user-specific map defines the target. The agent spends a fixed number of turns asking diagnostic questions, maintains a full-graph working estimate, and submits a mastery prediction for every node. The simulator can express partial knowledge and misconceptions, but receives hidden information only after the current question has been grounded. The final map is scored deterministically against the hidden map.

This design makes several competing explanations testable. Fixed and random policies reveal whether interaction alone is sufficient. Working-map and graph ablations test whether explicit state maintenance guides action. Simulator variations reveal whether rankings depend on one generative proxy. Evidence references distinguish accurate reconstruction supported by the dialogue from accurate guesses.

Our contributions are therefore methodological rather than empirical at this stage:

- We formulate active knowledge-state diagnosis as an identifiable target for functional user modeling, separating reconstruction quality from downstream task success.
- We derive a benchmark design in which the public graph, hidden map, turn budget, and visibility boundary each control a specific alternative explanation for apparent success.
- We specify a validation protocol that separates adaptivity, explicit state maintenance, simulator dependence, and transfer to human interaction instead of collapsing them into one leaderboard score.

The present paper establishes the construct, artifacts, and tests needed to evaluate these claims. It does not yet provide the empirical evidence required to conclude that any agent exhibits the target behavior. That distinction is deliberate: KnowAct is intended to make such a claim falsifiable before it is made.

2 Related Work

2.1 From Mental-State Inference to Functional Use

Machine-ToM benchmarks have made latent-state inference increasingly controlled and difficult. ToMi exposes shortcuts in false-belief question answering (Le et al., 2019); BigToM uses causal templates (Gandhi et al., 2023); and FANToM tests consistency across questions that depend on the same information asymmetry (Kim et al., 2023). Hi-ToM, OpenToM, and ToMBench broaden reasoning order, narrative variation, and the taxonomy of social-cognitive abilities (Wu et al., 2023; Xu et al., 2024; Chen et al., 2024b).

These advances strengthen the validity of inference from supplied evidence. They do not make evidence acquisition part of the tested policy. SimpleToM identifies a related boundary: explicitly inferring a mental state does not guarantee that a model will use it when predicting or judging behavior (Gu et al., 2024). KnowAct takes this boundary as a design requirement. A user model must affect the next diagnostic action, and the result of that process must remain directly scoreable.

Interactive social benchmarks test behavior in richer loops. SOTOPIA and AgentSense evaluate social goals, strategic communication, and private-information reasoning in multi-turn scenarios (Zhou et al., 2024; Mou et al., 2025). Applied-ToM agents explicitly insert mental-state hypotheses into social, persuasive, or software-engineering policies (Hwang et al., 2026; Wang et al., 2026; Zhou et al., 2026).

Their question is whether user modeling improves an external outcome. Ours is whether the user model itself is acquired accurately and adaptively. These are complementary claims: downstream success establishes usefulness, whereas direct reconstruction establishes what was learned about the user. Broader agent evaluation likewise shows that single-turn ability and final success can obscure multi-turn and trajectory-level differences (Wang et al., 2024; Ma et al., 2024).

2.2 Passive User Modeling and Active Diagnosis

Personalization benchmarks infer preferences and traits from user histories. LaMP evaluates whether profile information improves personalized outputs (Salemi et al., 2024); PERSON-AMEM asks whether models track profiles that evolve across long interaction histories (Jiang et al., 2025). In both cases, the history is evidence to be interpreted. The tested model does not decide which observation would most reduce uncertainty about the user.

Knowledge tracing provides the closest formal analogue. Bayesian and neural tracing methods estimate latent mastery from sequences of item responses (Corbett & Anderson, 1994; Piech et al., 2015; Ghosh et al., 2020). LongTutor further separates historical evidence acquisition, knowledge diagnosis, and adaptive teaching, revealing that strength at one stage need not transfer to the next (Li et al., 2026).

KnowAct changes who controls the evidence. The tested agent asks open-ended questions and is scored on the full latent state after a bounded number of observations. The benchmark is therefore an active diagnosis problem rather than next-response prediction over a fixed curriculum.

2.3 LLM-Assisted Knowledge-Graph Construction

LLM-based graph construction commonly decomposes extraction, schema induction, canonicalization, and verification. EDC and KGGen address schema scale and entity resolution (Zhang & Soh, 2024; Mo et al., 2025); PiVe, SAC-KG, GraphJudge, and GraphRefine add verification, filtering, or repair to improve factual graph quality (Han et al., 2024; Chen et al., 2024a; Huang et al., 2025; Kim et al., 2026).

Those methods primarily ask whether text has been converted into accurate entity-relation triples. In KnowAct, graph quality has a second consequence: it defines the variables over which a user is diagnosed. A source-faithful but non-diagnosable node, or a rubric whose

Table 1: Positioning of KnowAct relative to adjacent evaluation paradigms. “Active probes” means the tested model chooses what evidence to request.

Paradigm	Interactive	Explicit latent user state	Active probes	Direct state score
Narrative / conversational ToM	Limited	Partial	No	Usually QA
Social-agent arenas	Yes	Usually no	Yes	No
Personalization from histories	Usually no	Profile / preference	No	Sometimes
Knowledge tracing	Sequential	Mastery	Usually no	Prediction-based
KnowAct	Yes	Knowledge map	Yes	Yes

adjacent levels cannot be distinguished in dialogue, weakens the benchmark even if the graph is factually correct.

We therefore treat graph authoring as measurement design. Competency questions delimit the intended use of an ontology (Grüniger & Fox, 1995), while evidence-centered design connects domain claims to observations that can support them (Mislevy et al., 2003). LLMs propose source-traceable candidates, but graph constraints and domain review decide which concepts and relations become part of the scored hypothesis space. Independent verification is a control against self-approval, not a claim that additional agents are intrinsically better.

2.4 LLM-Based User Simulation

User simulation trades experimental control for behavioral fidelity. Agenda-based simulators are reproducible but narrow; LLM simulators are expressive but can invent goals, leak hidden state, or impose model-specific artifacts. DuetSim separates generation and verification to improve response quality (Luo et al., 2024). SimulatorArena instead evaluates the proxy assumption directly, comparing simulated behavior and assistant rankings with human interaction (Dou et al., 2025). Earlier simulator research makes the same conceptual point: fluency is not fidelity to task-relevant behavior (Shi et al., 2019).

KnowAct does not treat an LLM simulator as ground truth about human conversation. It uses simulation to hold a reviewed latent state fixed while agents choose evidence. Structural access controls limit what can be expressed, deterministic scoring prevents the simulator from judging success, and matched human episodes test whether conclusions transfer beyond the proxy. Simulator validation is therefore part of the benchmark claim, not an optional naturalness check.

3 The KnowAct Benchmark

KnowAct is organized around an attribution question: when an agent reconstructs a user accurately, what makes that performance evidence of active user modeling rather than prior knowledge, exhaustive questioning, or leakage? Each benchmark component removes one ambiguity. The graph fixes the hypothesis space, the hidden map fixes the target, the turn budget gives questions an opportunity cost, and the visibility boundary fixes what can count as evidence.

3.1 Measurement Target and Identifiability

We operationalize functional ToM as a conjunction of three observable abilities: infer a user state from interaction evidence, revise that state as evidence accumulates, and condition the next action on the current estimate. Testing only the first recovers a static inference task; testing only the last permits a generic policy with no explicit user model.

The construct is intentionally behavioral. KnowAct does not assume that an agent represents beliefs as humans do. It asks whether a structured estimate explains and supports the agent’s diagnostic behavior under controlled information access.

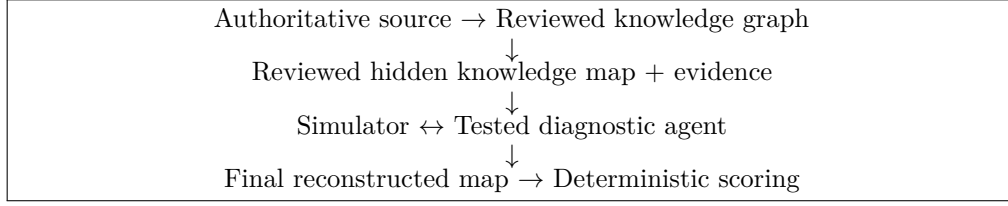


Figure 1: The KnowAct pipeline. Authoring produces reviewed hidden artifacts; the tested agent interacts only through the visible episode interface.

A shared hypothesis space. A benchmark domain is represented by an authored knowledge graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad (1)$$

where each node $v_i \in \mathcal{V}$ is a diagnosable knowledge unit and each edge $e_{ij} \in \mathcal{E}$ is a reviewed domain relation.

The graph is public to both agent and simulator. It removes domain discovery from the target capability and makes competing agents reason over the same variables. Nodes contain definitions, source locators, diagnostic goals, mastery rubrics, and diagnostic signals. Edges express domain structure; they never encode a particular user’s state.

An explicit latent target. For a graph with $N = |\mathcal{V}|$ nodes, the hidden map is

$$\mathcal{M}^* = \{s_i^*\}_{i=1}^N. \quad (2)$$

Each state includes mastery $m_i^* \in \{0, 1, \dots, 5\}$, evidence references, misconceptions, and unknowns. Levels are node-specific rubric positions, not a universal psychometric scale.

Making the target explicit allows direct reconstruction scoring, but it also limits the claim. The map is an operational description of knowledge relevant to the authored graph, not a complete psychological profile. Optional profile context controls conversational style; it is neither scored nor allowed to override node states.

A budgeted identification problem. An episode is specified by

$$\mathcal{E}p = (\mathcal{G}, \mathcal{M}^*, \mathcal{Z}, T, \rho, \sigma), \quad (3)$$

where $\mathcal{Z}$ is simulator-only evidence, T is the turn budget, ρ is the interaction rule, and σ is the scoring profile.

At turn t , the agent observes the public graph, visible history $\mathcal{H}_{t-1}$, and its working map $\widehat{\mathcal{M}}_{t-1}$. It chooses one diagnostic question

$$q_t \sim \pi_\theta(\mathcal{G}, \mathcal{H}_{t-1}, \widehat{\mathcal{M}}_{t-1}), \quad (4)$$

receives answer a_t , and updates its estimate

$$\widehat{\mathcal{M}}_t = U_\theta(\widehat{\mathcal{M}}_{t-1}, q_t, a_t). \quad (5)$$

After at most T turns, the agent submits a full-graph reconstruction $\widehat{\mathcal{M}}_T$. The hidden map, evidence, simulator traces, and scoring internals remain inaccessible throughout the episode.

The budget is what turns dialogue into active diagnosis. Without it, exhaustive coverage can erase differences among acquisition policies. Requiring a full-graph submission prevents an agent from reporting only easy or directly queried nodes. Adaptivity is then a counterfactual claim: after different answers to the same early probe, a policy with a functioning user model should have reason to choose different subsequent probes.

3.2 Constructing a Scorable Latent State

The graph is a measurement model. The initial domain is statistical learning, grounded in An Introduction to Statistical Learning with Applications in Python (James et al., 2023). Source grounding constrains what the graph may claim, but factual support alone is insufficient. Node granularity determines what distinctions the benchmark can observe: coarse nodes hide partial knowledge, while excessively fine nodes reward terminology rather than understanding.

Authors therefore fix the aspect, representative tasks, exclusions, and node budget before extraction; the initial configuration targets roughly 20 representative nodes and caps the graph at 24. LLMs generate candidates, but do not define the construct: source-linked extraction, global reconciliation, independent verification, and domain review determine what enters the scored space. This distinction is essential because definitions can be source-supported while rubrics and relation rationales remain pedagogical hypotheses. Fail-closed validation and immutable traces make both kinds of judgment inspectable. Appendix B specifies the complete workflow.

The map separates state from expression. A hidden map assigns one reviewed state to every graph node. The scored node-level state, simulator-only behavioral evidence, and optional style context remain separate artifacts. Consequently, fluency cannot substitute for mastery, and the simulator cannot invent evidence to justify a label. Review asks not only whether a map is complete and internally consistent, but whether its distinctions can plausibly become observable in dialogue.

Comparability is an artifact property. An immutable manifest binds the graph version, hidden map, turn budget, interaction rule, scoring profile, and pinned provider configuration. This prevents a change in model ranking from being silently attributed to a changed graph, user, or simulator.

3.3 User Simulation and Visibility Boundary

Simulation creates the benchmark’s central validity tension. A rigid state-query API would be controllable but would remove natural-language diagnosis; an unconstrained role-playing model would be natural but could invent knowledge, reveal labels, or answer from graph-wide context. KnowAct treats the simulator as an information boundary that must mediate between these objectives.

Ground before revealing hidden state. A grounder maps the question to a directly targeted node set $S_t \subseteq \mathcal{V}$ and rejects ungrounded, hidden-label, or compound requests. Because grounding sees the public graph and limited visible dialogue but no hidden state, the target is determined by the question’s meaning rather than by which private facts would make it easy to answer.

Reveal only what the question licenses. The context builder retrieves state and evidence only for S_t ; graph neighbors are not revealed merely because an edge exists. Thus, relations may support an agent’s inference without turning the simulator into a full-map oracle. An answer policy converts this restricted context into a de-identified blueprint of permitted claims and uncertainty boundaries, excluding mastery labels, hidden identifiers, and scoring fields.

Separate content fidelity from surface realization. The generator realizes the blueprint using only the visible dialogue needed for continuity and an optional style hint. The policy therefore controls what may be revealed, while generation controls how it is said. Separate tested-agent, simulator, and evaluation contexts keep hidden traces auditable but invisible. These controls establish non-leakage under the implemented boundary; human fidelity remains an empirical question tested in Section 4.

3.4 Active Diagnosis Agent Protocol

The protocol constrains what must be observable, not how an agent must reason internally. Agents may use prompting, probabilistic inference, planning, or learned policies. They share an external state and action interface so that differences in performance can be interpreted under the same evidence and budget.

The working map is an instrument. The tested agent maintains an assessment for every node:

$$\hat{s}_{i,t} = (\hat{m}_{i,t}, c_{i,t}, n_{i,t}, R_{i,t}), \quad (6)$$

where $\hat{m}_{i,t} \in \{\text{unknown}, 0, \dots, 5\}$ is assessed mastery, $c_{i,t}$ is confidence, $n_{i,t}$ is an optional note, and $R_{i,t}$ contains supporting visible turns.

All nodes begin as unknown. Non-unknown assessments should cite visible evidence; unsupported estimates remain scoreable but are reported separately. The schema is not claimed to be cognitively privileged. It makes state maintenance inspectable, so a coherent update trajectory can be distinguished from a late guess.

Actions expose the closed loop. The runtime exposes only benchmark-relevant operations:

- read the authored episode graph or current working map;
- update one node assessment with visible support;
- ask one diagnostic question;
- finalize the reconstructed map.

These operations make two links observable: answers should change the working map, and the working map should influence later probes. Restricting the interface also prevents a policy from bypassing the intended loop through generic access to benchmark artifacts.

The resulting loop is intentionally underspecified: interpret an answer, revise the working map, and select a probe expected to reduce decision-relevant uncertainty. KnowAct neither prescribes an inference algorithm nor assumes optimal experimental design; it tests whether later choices use the information the agent claims to maintain.

3.5 Evaluation Metrics

No single metric establishes the full infer-update-act claim. The primary score measures endpoint reconstruction because this target is deterministic and common to all agents. Process measures then test whether that endpoint is supported by visible evidence and acquired efficiently.

Endpoint reconstruction. Let the final predicted mastery for node i be $\hat{m}_i$. For submitted levels 0 through 5, the node loss is squared ordinal distance:

$$d(\hat{m}_i, m_i^*) = (\hat{m}_i - m_i^*)^2. \quad (7)$$

An unknownprediction receives the fixed maximum penalty of 36. The episode score is

$$D_{\text{mastery}} = \frac{1}{N} \sum_{i=1}^N d(\hat{m}_i, m_i^*), \quad (8)$$

where lower is better. The score is deterministic and requires no evaluator model.

Squared ordinal distance encodes two judgments: adjacent mistakes are less serious than endpoint mistakes, and large errors should grow more than linearly. An unknownprediction is penalized more heavily than any valid level error so that selective non-commitment cannot improve the score. These are benchmark conventions, not claims that the six levels form a universal psychometric interval scale; interpretation remains anchored to each node’s rubric.

Evidence and acquisition diagnostics. Secondary metrics expose explanations hidden by the mean endpoint score:

- exact mastery match, signed error, and missing-prediction rate;
- unsupported inference rate based on absent visible support;
- graph coverage and reconstruction quality by turn count;
- redundant or ungrounded question rate;
- information gain per turn for agents that expose calibrated beliefs.

Misconceptions and free-text unknowns remain available for qualitative analysis but are excluded from the initial automatic score. This gives up some semantic richness to avoid making an additional LLM judge part of the primary result. The experimental comparisons in Section 4 determine which combinations of these signals support stronger claims about adaptivity.

4 Experimental Evaluation

The benchmark supports progressively stronger claims, and the experiments must keep them separate. Accurate final maps establish reconstruction; an advantage over non-adaptive controls establishes that question choice matters; state and graph ablations test why it matters; simulator and human comparisons test whether the conclusion survives outside one proxy environment.

Empirical runs are not yet complete. We therefore state the comparisons and falsification criteria before reporting results. This section is an evaluation protocol, not evidence that the proposed construct has already been validated.

4.1 Claims and Falsification Criteria

We organize evaluation around four hypotheses:

1. H1—adaptive acquisition: under equal turn budgets, state-conditioned policies reconstruct hidden maps more accurately than fixed and random diagnostic policies.
2. H2—explicit state use: removing the working map or graph access degrades either reconstruction or early-turn efficiency.
3. H3—simulator robustness: comparative conclusions persist across simulator model, grounding, answer policy, and profile style.
4. H4—human transfer: simulator answers preserve task-relevant human behavior closely enough that agent rankings agree with matched human-user episodes.

Each hypothesis blocks a different over-interpretation. A low reconstruction loss does not establish H1 if a fixed script performs equally well. H1 does not establish H2 if adaptivity comes from unstructured dialogue history alone. H1 and H2 do not establish external validity if rankings reverse under another simulator or with human users. Failure of H3 or H4 would narrow the benchmark claim rather than merely lower a score.

4.2 Paired Comparative Design

The initial study is a construct-validation experiment in statistical learning, not a test of cross-domain generality. The unit of comparison is a reviewed episode. Policies receive the same public graph, hidden-user instance, budget, interaction rule, and scoring profile; repeated stochastic runs remain nested within episodes.

Three controls give H1 its interpretation. A fixed-question policy controls for a reasonable diagnostic script, a random policy controls for interaction volume, and a simple LLM policy tests whether generic conversational adaptation is sufficient. Structured policies add an explicit working map, graph-aware planning, or an information-gain objective. Beating a

no-interaction prior shows that dialogue contains evidence; only a paired advantage over fixed and random policies shows that selecting evidence helps.

The primary analysis reports mastery distance, exact match, missing predictions, and unsupported inference at matched budgets. Final-turn performance tests reconstruction, whereas turn-indexed curves test acquisition efficiency. Per-node signed errors reveal calibration patterns hidden by an aggregate score.

4.3 Mechanism and Validity Tests

Working-map and graph ablations target H2. Simulator, grounding, answer-policy, and profile-style variations target H3; both absolute changes and rank reversals are reported because only the latter directly threaten comparative validity. Matched human episodes target H4 at the turn level through state fidelity and at the episode level through ranking transfer.

Graph review is a prerequisite validity check because the graph defines the scored variables. Two domain reviewers assess source support, granularity, diagnostic usefulness, and relation validity. A matched one-shot authoring comparison tests only whether the staged workflow reduces unsupported or duplicate candidates and review effort; it does not establish a uniquely correct ontology.

Appendix D specifies artifact bindings, review judgments, statistical units, ablations, and error categories. Empirical tables will replace the placeholders below once the protocol has been executed.

5 Discussion and Conclusion

5.1 What a KnowAct Result Would Mean

A benchmark score is evidence only relative to the explanations that the design excludes. Low reconstruction error alone is compatible with several stories: the agent inferred the user from dialogue, exploited a strong population prior, received leaked information, or guessed well. KnowAct strengthens interpretation by pairing endpoint accuracy with controlled access, evidence references, and non-adaptive comparisons.

The resulting claims form a hierarchy:

1. reconstruction: the final map matches the reviewed map;
2. active acquisition: adaptive policies outperform matched non-adaptive controls;
3. state use: working-map or graph ablations identify a mechanism for that advantage;
4. external validity: the advantage is robust across simulators and transfers to human interaction.

Evidence for a lower level does not entail a higher one. In particular, accurate reconstruction is not yet functional ToM if the same result is achieved by a fixed script. Conversely, failure to transfer to humans would indict the simulator proxy, not necessarily the internal consistency of the reconstruction task.

The interaction can be viewed as active information acquisition over a structured latent state. It differs from pool-based active learning because probes are open-ended utterances whose meaning must be grounded. It differs from conventional adaptive testing because the item bank is not fixed or psychometrically calibrated. This expressivity is precisely why provenance, grounding, and proxy validation are constitutive parts of the measurement argument.

Even a fully validated result would remain diagnostic rather than task-complete. KnowAct asks whether an agent can acquire a useful user model; it does not yet establish that the model improves teaching, recommendation, or collaboration. Connecting reconstruction to downstream benefit is a separate claim and a natural next benchmark layer.

5.2 Limitations and Threats to Validity

The target is authored, not discovered. The graph and rubrics determine which distinctions count as knowledge. Expert review and source provenance make that choice defensible and inspectable, but not uniquely correct. Rankings may change under another granularity, relation set, or rubric design.

Simulator fidelity. A simulator can be perfectly consistent with a reviewed map while expressing partial knowledge unlike a person. Leakage tests establish access control, not behavioral realism. Matched human studies must test diagnostic value and ranking transfer rather than relying on naturalness alone.

Construction circularity. Related model families may propose graph artifacts, simulate users, and serve as tested agents. Shared inductive biases could make the benchmark easier for some models. Source grounding, human review, model-family diversity, deterministic fixtures, and human validation reduce but cannot remove this dependence.

Simplified knowledge state. A static L0–L5 state cannot capture context dependence, procedural fluency, confidence, forgetting, or learning during dialogue. Freezing state makes the reconstruction target identifiable, but it also turns explanations into observations rather than teaching events. Dynamic diagnosis will require a transition model and a different causal interpretation.

Initial domain scope. The first release focuses on statistical learning and a small authored graph. It can validate the protocol but not cross-domain generality. Additional domains must vary source style, graph topology, concept granularity, and the kinds of misconceptions that can be expressed.

Operational claims. Success would demonstrate behavior compatible with structured user modeling under benchmark constraints. It would not establish human-like beliefs, consciousness, or a psychologically faithful ToM mechanism. The term functional ToM names the tested role of the representation, not its ontological status.

5.3 Conclusion and Outlook

Evaluating user-aware agents requires more than placing them in a conversation. The evaluator must know what user state is at stake, make evidence acquisition consequential, and distinguish accurate inference from leakage, priors, and generic interaction skill.

KnowAct offers one operationalization of these requirements. It turns a reviewed knowledge map into an explicit latent target, lets agents choose natural-language probes under a budget, and exposes the link from visible evidence to working state and subsequent action. The simulator boundary and deterministic score make this loop auditable without claiming that simulation is equivalent to human interaction.

The immediate next step is not to broaden the claim but to test it: adaptive controls, mechanism ablations, simulator variation, and matched human episodes must establish which level of the evidence hierarchy KnowAct can support. Only then should the benchmark be extended to dynamic learning, downstream assistance, and additional domains.

Ethics Statement

User modeling can improve tutoring, accessibility, and collaborative assistance. It can also enable manipulation, surveillance, or unjustified inference about individuals.

KnowAct uses synthetic or explicitly reviewed benchmark profiles. It should not be interpreted as permission to infer sensitive attributes from real users without informed consent.

Derived systems should minimize retained personal data, expose uncertainty, and let users inspect or correct inferred states. Knowledge estimates should not determine high-stakes outcomes without human oversight.

Benchmark releases should document source provenance and licenses. Profile contexts, evidence records, and released traces must not contain identifiable personal information.

References

- Hanzhu Chen, Xu Shen, Qitan Lv, Jie Wang, Xiaoqi Ni, and Jieping Ye. SAC-KG: Exploiting large language models as skilled automatic constructors for domain knowledge graph. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 4345–4360. Association for Computational Linguistics, 2024a. doi: 10.18653/v1/2024.acl-long.238.
- Zhuang Chen, Jincenzi Wu, Jinfeng Zhou, Bosi Wen, Guanqun Bi, Gongyao Jiang, Yaru Cao, Mengting Hu, Yunghwei Lai, Zexuan Xiong, and Minlie Huang. ToMBench: Benchmarking theory of mind in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 15959–15983. Association for Computational Linguistics, 2024b. doi: 10.18653/v1/2024.acl-long.847.
- Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1994.
- Yao Dou, Michel Galley, Baolin Peng, Chris Kedzie, Weixin Cai, Alan Ritter, Chris Quirk, Wei Xu, and Jianfeng Gao. SimulatorArena: Are user simulators reliable proxies for multi-turn evaluation of AI assistants? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 35212–35290. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1786.
- Kanishk Gandhi, Jan-Philipp Fränken, Tobias Gerstenberg, and Noah D. Goodman. Understanding social reasoning in language models with language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Aritra Ghosh, Neil Heffernan, and Andrew S. Lan. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 2330–2339. ACM, 2020.
- Michael Grüninger and Mark S. Fox. The role of competency questions in enterprise engineering. In *Benchmarking—Theory and Practice*, pp. 22–31. Springer, 1995. doi: 10.1007/978-0-387-34847-6_3.
- Yuling Gu, Oyvind Tafjord, Hyunwoo Kim, Jared Moore, Ronan Le Bras, Peter Clark, and Yejin Choi. SimpleToM: Exposing the gap between explicit ToM inference and implicit ToM application in LLMs. *arXiv preprint arXiv:2410.13648*, 2024.
- Jiuzhou Han, Nigel Collier, Wray Buntine, and Ehsan Shareghi. PiVe: Prompting with iterative verification improving graph-based generative capability of LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 6702–6718. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.400.
- Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. Can LLMs be good graph judge for knowledge graph construction? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 10929–10948. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.554.
- EunJeong Hwang, Yuwei Yin, Giuseppe Carenini, Peter West, and Vered Shwartz. Infusing theory of mind into socially intelligent LLM agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 11327–11360. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.551.
- Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning: with Applications in Python*. Springer, 2023.

- Bowen Jiang, Zhuoqun Hao, Young Min Cho, Bryan Li, Yuan Yuan, Sihao Chen, Lyle Ungar, Camillo Jose Taylor, and Dan Roth. Know me, respond to me: Benchmarking LLMs for dynamic user profiling and personalized responses at scale. In *Conference on Language Modeling*, 2025.
- Donghyun Kim, Hyeongjun Yang, Seokju Hwang, Kyong-Ho Lee, and Chanhee Lee. LLMs as knowledge graph refiners: Mitigating factual inconsistencies in generative knowledge extraction. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 29358–29378. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1353.
- Hyunwoo Kim, Melanie Sclar, Xuhui Zhou, Ronan Bras, Gunhee Kim, Yejin Choi, and Maarten Sap. FANToM: A benchmark for stress-testing machine theory of mind in interactions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 14397–14413. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.890.
- Matthew Le, Y-Lan Boureau, and Maximilian Nickel. Revisiting the evaluation of theory of mind through question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 5872–5877. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1598.
- Ning Li, Zheng Zhang, Zhenya Huang, Rui Li, Yi Zhan, Yinbo Luo, Qi Liu, and Enhong Chen. LongTutor: Benchmarking large language models for long-term personalized tutoring. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 29712–29737. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1371.
- Xiang Luo, Zhiwen Tang, Jin Wang, and Xuejie Zhang. DuetSim: Building user simulator with dual large language models for task-oriented dialogues. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pp. 5414–5424. ELRA and ICCL, 2024.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Robert J. Mislevy, Russell G. Almond, and Janice F. Lukas. A brief introduction to evidence-centered design. Technical Report RR-03-16, Educational Testing Service, 2003.
- Belinda Mo, Kyssen Yu, Joshua Kazdan, Proud Mpala, Lisa Yu, Charilaos Kanatsoulis, and Sanmi Koyejo. KGen: Extracting knowledge graphs from plain text with language models. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- Xinyi Mou, Jingcong Liang, Jiayu Lin, Xinnong Zhang, Xiawei Liu, Shiyue Yang, Rong Ye, Lei Chen, Haoyu Kuang, Xuanjing Huang, and Zhongyu Wei. AgentSense: Benchmarking social intelligence of language agents through interactive scenarios. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4975–5001. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.257.
- Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. LaMP: When large language models meet personalization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 7370–7392. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.399.

- Weiyang Shi, Kun Qian, Xuewei Wang, and Zhou Yu. How to build user simulators to train RL-based dialog systems. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp. 1990–2000. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1206.
- Ruirui Wang, Haoran Zhang, Tian Lan, Zehua Duo, Jiang Li, Guanglai Gao, and Xiangdong Su. ToMELP: A theory-of-mind benchmark for route-controlled persuasion under the elaboration likelihood model. In Findings of the Association for Computational Linguistics: ACL 2026, pp. 21320–21338. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1072.
- Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback. In International Conference on Learning Representations, 2024.
- Yufan Wu, Yinghui He, Yilin Jia, Rada Mihalcea, Yulong Chen, and Naihao Deng. Hi-ToM: A benchmark for evaluating higher-order theory of mind reasoning in large language models. In Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 10691–10706. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.717.
- Hainiu Xu, Runcong Zhao, Lixing Zhu, Jinhua Du, and Yulan He. OpenToM: A comprehensive benchmark for evaluating theory-of-mind reasoning capabilities of large language models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, pp. 8593–8623. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.466.
- Bowen Zhang and Harold Soh. Extract, define, canonicalize: An LLM-based framework for knowledge graph construction. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 9820–9836. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.548.
- Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haoqi Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. SO-TOPIA: Interactive evaluation for social intelligence in language agents. In International Conference on Learning Representations, 2024.
- Xuhui Zhou, Valerie Chen, Zora Zhiruo Wang, Graham Neubig, Maarten Sap, and Xingyao Wang. TOM-SWE: User mental modeling for software engineering agents. In Proceedings of the 43rd International Conference on Machine Learning, 2026.

A Implementation-Oriented Artifact Schemas

The abbreviated schemas below summarize the benchmark contracts. Field names may evolve, but the separation among public graph, hidden map, visible observations, and final reconstruction should remain stable.

A.1 Knowledge Node

```
{
  "id": "active_learning",
  "name": "Active Learning",
  "type": "concept",
  "definition": "...",
  "source_locators": [...],
  "diagnostic_goal": "...",
  "levels": {"L0": "...", ..., "L5": "..."},
  "diagnostic_signals": [...],
  "simulator_behavior": "..."}
}
```

A.2 Hidden User State

```
{
  "node_id": "active_learning",
  "mastery_level": "L2",
  "evidence_refs": ["ev_104"],
  "misconceptions": [...],
  "unknowns": [...]
}
```

A.3 Agent Working Assessment

```
{
  "node_id": "active_learning",
  "assessed_mastery_level": "L2",
  "diagnostic_confidence": "medium",
  "assessment_note": "Understands the motivation but not
    query-strategy trade-offs.",
  "supporting_turn_ids": ["turn_03"]
}
```

B Benchmark Construction Details

B.1 Graph-Authoring Workflow

The graph-authoring workflow is evidence-first and fail-closed:

1. Specification. Authors manually prepare one UTF-8 Markdown source and declare an aspect name and description, representative user tasks, exclusions, a soft target node count, and a hard maximum.
2. Source integrity. The local catalog records the Markdown file’s size and SHA-256 hash; invalid extensions, undecodable content, blank files, and files over the configured size limit are rejected.
3. Deterministic segmentation. Heading-aware, non-overlapping segments retain their source path, heading path, character count, and source locator.
4. Local extraction. Each segment is paired with the same scope. The extractor emits thin node drafts with a short exact evidence excerpt, and code verifies that each excerpt occurs in its source segment after whitespace normalization.
5. Global reconciliation and selection. One global step merges duplicates, adjusts granularity, records supporting draft and segment identifiers, and selects representative skeletons near the target without exceeding the maximum. It may return fewer nodes rather than fill the graph with weak candidates.
6. Independent verification. A fresh context assigns one keep/remove decision to every skeleton based on source support, scope fit, and diagnostic value. Only supported, in-scope, high- or medium-value nodes continue.
7. Diagnostic enrichment. Retained skeletons receive goals, mastery rubrics, diagnostic signals, and simulator guidance.
8. Relation authoring and validation. A precision-first step proposes only typed relations with explicit rationales. Deterministic checks reject invalid endpoints, self-loops, duplicate identifiers, and canonicalization errors.
9. Human review and promotion. Candidate node and edge lists remain outside the evaluation runtime until reviewers inspect them and an author explicitly promotes an immutable version.

The relation vocabulary includes `part_of`, `prerequisite_for`, `supports`, and `contrasts_with`. Each label has tests, examples, direction rules, and allowed evidence.

prerequisite_for denotes a direct dependency: lacking the source concept should normally block stable performance at higher rubric levels of the target. Transitive implications are not emitted as additional edges.

part_of denotes structural composition. supports denotes a useful but nonessential explanatory or transfer relation. contrasts_with is stored in a canonical symmetric form.

Rubrics and instructional prerequisite judgments are candidate pedagogical operationalizations, not automatically treated as textbook facts. Their validity is established by human review rather than by structural validation alone.

B.2 Agent Roles and Decision Independence

A deterministic orchestrator invokes five bounded model roles: local extractor, global reconciler/selector, independent skeleton verifier, rubric author, and relation proposer. Deterministic code handles segmentation, parsing, evidence membership, budget/provenance checks, and final schema/graph validation.

Roles are logical separation, not a claim that more agents improve quality. Independent prompts and fresh contexts prevent a proposer from validating its own rationale; a different model can be used when resources permit.

Local extraction requests may run concurrently, but results and traces are assembled in source order. Reconciliation is global because the initial narrow slice is intentionally small enough to compare all drafts under one aspect and node budget.

B.3 Bounded Error Recovery

The current v1 implementation uses fail-fast, stage-level validation rather than autonomous semantic repair. A provider failure, malformed response, missing or mismatched evidence excerpt, invalid reconciliation provenance, verifier contract violation, node-budget overflow, rubric mismatch, or invalid edge stops downstream execution.

When artifact writing is enabled, completed intermediates and per-step raw/parser traces remain available with a structured workflow log. The benchmark author inspects the failing checkpoint, source evidence, declared scope, and model output, then corrects the input or prompt and starts a new run. The same agent is not allowed to silently rewrite and approve its own invalid item.

This policy favors visible failure over silent corruption. Automatic retry, semantic repair, coverage recovery, and resumable checkpoints are possible extensions, but are not claimed as properties of the current workflow.

B.4 Expert Review Protocol

For the initial approximately 20-node graph, two domain experts review every candidate node and edge independently. They resolve disagreements by discussion; a third expert can adjudicate unresolved cases.

Reviewers are recruited from instructors, experienced teaching assistants, and researchers in the target subject. Teaching history and a held-out calibration task establish eligibility.

Review captures item-level accept, edit, and reject decisions with rationales. Node judgments cover source support, aspect fit, granularity, rubric quality, and diagnostic usefulness; edge judgments cover endpoints, type, direction, and rationale. We record reviewer qualifications, review time, and inter-reviewer agreement.

Promotion requires complete machine-checkable provenance, zero unresolved structural errors, and explicit reviewer confirmation. Model verification reduces review load but never replaces expert judgment.

B.5 Knowledge-Map Review

Reviewers verify that each candidate map covers every node and that each state is supported by behaviorally realizable evidence.

They also check consistency among mastery, misconceptions, unknowns, profile context, and graph structure. Promotion creates an immutable reviewed snapshot.

B.6 Manifest-Bound Runtime Contexts

The runtime maintains three contexts:

- Tested-agent context: public graph, interaction rules, visible transcript, and working map.
- Simulator context: grounded hidden states, evidence, limited visible dialogue, and optional style context.
- Evaluation context: hidden map, final reconstruction, and deterministic scoring configuration.

An episode manifest fixes the artifact versions and execution settings shared by these contexts. Debug traces and hidden policy outputs never enter the tested-agent context.

C Simulator and Agent Interface Details

C.1 Simulator Stage Contracts

The question grounder receives the public graph, the current question, and limited visible history. It returns grounded node identifiers or a structured rejection reason.

The answer policy receives only the grounded hidden context. It returns a de-identified blueprint that constrains claims, uncertainty, misconceptions, cues, and prohibited overstatements.

The generator receives the blueprint and surface-style context. It produces the user-visible answer without access to raw mastery labels, map identifiers, evidence identifiers, or scoring fields.

C.2 Tested-Agent Actions

The tested agent can read the public graph and its working map, update a node with visible support, ask one grounded question, or finalize.

Each mutation and question is recorded in the episode trace. Exact prompt templates, structured schemas, and provider-specific settings should accompany the released benchmark artifact.

D Detailed Experimental Protocol

D.1 Artifact and Run Binding

Every reported run identifies the graph and map versions, turn budget, interaction rule, scoring profile, simulator configuration, tested-agent model, provider settings, and execution configuration. Reported graph versions link to the source hash, declared aspect scope, extraction evidence, reconciliation and verification decisions, workflow traces, and human-review record. Development fixtures without this chain are excluded from benchmark claims.

Agents receive identical public artifacts and episode constraints. Hidden maps, simulator evidence, policy traces, and scoring internals remain outside the tested-agent context. Repeated stochastic runs are nested within episodes; confidence intervals and paired tests use the episode as the independent unit.

D.2 Policy Controls and Analyses

The fixed-question agent follows a predefined diagnostic order. The random agent samples valid targets under the interaction constraints. The simple LLM agent chooses questions and updates its map through the same semantic interface as structured agents.

The main analysis reports mastery distance, exact match, missing predictions, unsupported inference, and per-node signed error. Turn-indexed curves distinguish early information acquisition from final reconstruction. Working-map, graph-relation, and evidence-reference ablations test state maintenance, graph-mediated generalization, and traceability.

D.3 Graph Quality Check

Two statistical-learning reviewers independently inspect every node and edge, resolving disagreement by discussion or third-party adjudication. Node judgments cover source support, scope fit, granularity, and diagnostic usefulness. Edge judgments cover endpoints, relation type, direction, and rationale. Acceptance, edit, rejection, agreement, and review-time statistics are reported separately from deterministic schema checks.

The same source and scope are processed with a one-shot extraction prompt and the staged evidence-and-verifier workflow while holding model and sampling settings fixed. Reviewer acceptance, unsupported-node rate, duplication, and review time provide a bounded workflow comparison.

D.4 Simulator, Human, and Error Analyses

Robustness checks vary simulator model, grounding implementation, answer policy, and profile style while holding maps and budgets fixed. Matched human episodes compare state fidelity, diagnostic usefulness, naturalness, leakage, agent rankings, and adaptivity effects.

Error categories include poor grounding, hidden-state leakage, missed evidence, stale working-map state, redundant probes, unsupported graph propagation, and premature finalization. Representative trajectories connect each error to both the map update and the subsequent action.